Dealflow

USER GUIDE

Self-hosting and install

Business development CRM

Version 1.2.0 · 10 November 2026
dealflow.umev.co.uk

Contents

What you need 3

What runs 3

How to install 4

Campaign email 4

Reverse proxy routes 6

How to sign in the first time 6

How to update 6

Backups and restore 6

Logs 7

Privacy defaults 7

Self-hosting and install

This page is for the person who runs the Dealflow server. Dealflow is self-hosted and single-company: one install serves one company, and your data stays on your own machine.

The full technical references are `docs/deploy.md` and `docs/reverse-proxy.md` in the repository. This page is the short version.

What you need

- A Linux server with Docker and the Docker Compose plugin.
- A domain for the app (for example `crm.example.com`) and a second one for file storage (for example `s3.crm.example.com`), both pointing at the server.
- A reverse proxy that handles HTTPS (for example Caddy or nginx). Dealflow does not include one.
- An SMTP account for the emails Dealflow sends (sign-in links, invites, notifications, quotes). Campaigns can use the same account or a separate bulk provider (see Campaign email).
- Optionally, an OpenRouter API key for the AI features.

What runs

Service	What it does
`app`	The Dealflow web app
`ws`	Live updates in the browser
`worker`	Background jobs: mail sync, scheduled email, automations, reminders, backups
`postgres`	The database
`minio`	File storage (attachments, imports, photos)
`migrate`	Runs once on each start to update the database

How to install

1. Get the code onto the server and copy the example settings:

```
cp .env.example .env
```

2. Fill in `.env`. The values that matter most:

Setting	What to put
`BASE_URL`	Your app address, for example `https://crm.example.com`
`NEXT_PUBLIC_WS_URL`	`wss://crm.example.com/_ws`
`MINIO_ENDPOINT`	Your storage address, for example `https://s3.crm.example.com`
`SEED_ADMIN_EMAIL`	The email of the first admin
`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`	Your outgoing mail account
`OPENROUTER_API_KEY`, `OPENROUTER_MODEL`	Your AI key and model (leave the key empty to run without AI)
`BASE_CURRENCY`	Your company currency, for example `GBP`

3. Generate the secrets:

```
openssl rand -base64 32   # TOKEN_ENCRYPTION_KEY
openssl rand -hex 32      # WS_TICKET_SECRET
openssl rand -hex 16      # MINIO_ACCESS_KEY and MINIO_SECRET_KEY
```

4. Start everything:

```
docker compose up -d --build
```

5. Point your reverse proxy at the three routes below, then check the app is healthy:

```
curl -fsS https://crm.example.com/api/health
```

Result: `{"ok":true}`.

Campaign email

Campaigns send through a separate account so a large marketing send does not share the reputation of your sign-in and notification mail. Leave `MARKETING_SMTP_HOST` empty to fall back to the `SMTP_*` account above.

Setting	What to put
`MARKETING_SMTP_HOST`, `MARKETING_SMTP_PORT`, `MARKETING_SMTP_SECURE`, `MARKETING_SMTP_USER`, `MARKETING_SMTP_PASS`	Your bulk sending account. Port 465 with `MARKETING_SMTP_SECURE=true` is the common choice.
`CAMPAIGN_SEND_RATE_PER_MINUTE`	The hard ceiling on messages per minute. The team can lower it in **Settings › Campaigns**, never raise it above this value.
`CAMPAIGN_BATCH_SIZE`	How many recipients one worker tick takes. 50 is the default; lower it if your provider is strict.
`CAMPAIGN_DOI_DAILY_LIMIT`	The install-wide cap on double opt-in emails per UTC day. 500 is the default.
`CAMPAIGN_DKIM_DOMAIN`, `CAMPAIGN_DKIM_SELECTOR`, `CAMPAIGN_DKIM_PRIVATE_KEY` or `CAMPAIGN_DKIM_PRIVATE_KEY_FILE`	Optional DKIM signing. Set the domain, the selector and one key field. Signing turns on only when all of them are present.
`CAMPAIGN_BOUNCE_IMAP_HOST`, `CAMPAIGN_BOUNCE_IMAP_PORT`, `CAMPAIGN_BOUNCE_IMAP_USER`, `CAMPAIGN_BOUNCE_IMAP_PASS`	Optional bounce mailbox. Set all four to collect asynchronous bounces and complaints; leave them empty to disable the job. Port 993 is the secure default.

The **From name**, **From email**, postal address and the consent defaults are set by an admin in **Settings › Campaigns**, not in `.env`. The first sender still has to open its verification email before the account can send.

Deliverability checklist

1. Use a sending domain (or subdomain) you control. Do not send campaigns from a free mailbox.
2. Publish SPF for the sending provider, DKIM for the selector in `CAMPAIGN_DKIM_SELECTOR`, and a DMARC record. Start DMARC with `p=none`, read the reports, then move to `p=quarantine` and `p=reject` once the legitimate sources are aligned.
3. Sign in to your bulk provider and verify the domain there too. The provider's own DKIM and SPF records must be present even when Dealflow signs the message.
4. Put the provider credentials in `MARKETING_SMTP_*`, restart the stack, then use **Settings › Campaigns › Domain check** to confirm what public DNS returns.
5. Warm up slowly: send to your most engaged contacts first, keep `CAMPAIGN_SEND_RATE_PER_MINUTE` low, and increase it over several days. A new domain that starts with a large list is the fastest way to land in spam.
6. Configure the bounce mailbox. It catches bounces that arrive after the send, marks hard-bounced addresses as suppressed and keeps the list clean.
7. Keep the postal address and the unsubscribe link in every campaign. They are required, and the one-click unsubscribe header depends on them.
8. After changing any of these values, run `docker compose up -d` so the app and worker pick them up.

For the team's day-to-day instructions, see Campaigns.

Reverse proxy routes

Public address	Send to	Notes
`https://crm.example.com`	`127.0.0.1:3000`	The app, including `/api/*`
`https://crm.example.com/_ws`	`127.0.0.1:8080`	Live updates (WebSocket). Forward the path and query unchanged.
`https://s3.crm.example.com`	`127.0.0.1:9000`	File uploads and downloads. Must match `MINIO_ENDPOINT` exactly.

Keep the app, ws and storage ports bound to `127.0.0.1` so only your proxy can reach them. A working Caddy example is in `docs/reverse-proxy.md`.

How to sign in the first time

1. Go to your app address, choose **Email link**, and enter the `SEED_ADMIN_EMAIL` address.
2. Open the link in the email and set a password.

If email is not working yet, create a one-time admin link on the server instead:

```
docker compose exec app node dist/bootstrap-link.mjs
```

Then invite everyone else from **Settings › Users** (see Admin).

How to update

```
git pull
docker compose up -d --build
```

Database changes are applied automatically before the app restarts.

Backups and restore

Dealflow makes a daily backup of the database and a copy of stored files, kept for a set number of days. Settings: `BACKUP_ENABLED`, `BACKUP_DIR` and `BACKUP_KEEP`. Admins can see the status and run an extra backup in **Settings › Backups**.

To restore, on the server, with the app stopped:

```
docker compose stop app worker ws
docker compose run --rm -v ./backups:/restore worker \
  node dist/backup-restore.mjs /restore/{backup file name}.tar.gz --files --yes
docker compose start app worker ws
```

Without `--files` only the database is restored. Copy the backup folder somewhere off the server too, so a disk failure cannot take both.

Logs

```
docker compose logs -f app      # or ws, worker
```

Privacy defaults

Usage telemetry and the update check are off by default (`DISABLE_TELEMETRY=true`, `DISABLE_UPDATE_CHECK=true`). Nothing is sent anywhere unless you turn it on, apart from the services you configure: your mail server, your AI provider, and any enrichment provider an admin enables.